WETRANSACT DOCS

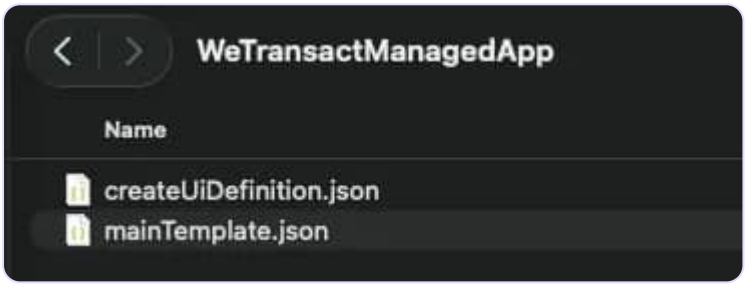
Build Managed Application Offer

docs.wetransact.io/build-managed-application-offer

🟣 Building the offer: the deployment package

To create a Managed Application offer you first build your solution in a repeatable deployment format. Azure uses ARM templates to define the infrastructure deployed into the customer's environment.

You prepare a **deployment package** – a .zip file containing two files at the root:



Warning: ***The deployment package must not contain binaries such as VM images. Reference all images from Marketplace instead.***

🟣 mainTemplate.json

An ARM template defining every Azure resource that makes up your application – virtual machines, storage accounts, databases, networking and so on. This template runs in the customer's subscription at deployment time.

- **File name casing matters.** The file must be named exactly **mainTemplate.json**, at the root of the zip.
- **Validate before uploading.** Run the ARM Template Test Toolkit (ARM TTK) against the package. It checks the template, the parameters and the UI definition against Marketplace requirements.
- **Start from working samples.** There are public repositories with hundreds of samples: Azure managed app samples on GitHub, the sample projects index and the broader Azure quickstart templates.

🟣 createUiDefinition.json

The UI definition file drives the interface your customer sees in the Azure portal during deployment. It lets you ask for configuration inputs – instance name, sizing, admin credentials, optional settings – in a wizard. The outputs of this form map one-to-one to the parameters of **mainTemplate.json.**

- **File name casing matters.** The file must be named exactly **createUiDefinition.json**, at the root of the zip.
- **Preview it in the sandbox.** Paste your file into the Create UI Definition Sandbox to preview the exact portal experience without deploying anything. Guide: Test your portal interface.
- **Reference docs:**createUiDefinition overview with all supported elements and functions.

Tip: ***Microsoft's free Mastering the Marketplace – Azure Managed Applications track has hands-on labs and videos covering the whole package-building process.***

🟣 Working example

The end result of development is two JSON files, zipped together and uploaded in the WeTransact portal. Depending on what you deploy, these files range from trivial to very large. The example below deploys a single storage account with one blob container.

🟣 mainTemplate.json

1{ 2 "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#", 3 "contentVersion": "1.0.0.0", 4 "parameters": { 5 "storageAccountType": { 6 "type": "string", 7 "defaultValue": "Standard_LRS", 8 "allowedValues": ["Standard_LRS", "Standard_GRS", "Premium_LRS"] 9 }, 10 "location": { 11 "type": "string", 12 "defaultValue": "[resourceGroup().location]" 13 }, 14 "containerName": { 15 "type": "string", 16 "defaultValue": "demo" 17 } 18 }, 19 "variables": { 20 "storageAccountName": "[concat('demo', uniqueString(resourceGroup().id))]" 21 }, 22 "resources": [23 { 24 "type": "Microsoft.Storage/storageAccounts", 25 "apiVersion": "2025-06-01", 26 "name": "[variables('storageAccountName')]", 27 "location": "[parameters('location')]", 28 "sku": { "name": "[parameters('storageAccountType')]" }, 29 "kind": "StorageV2", 30 "properties": { 31 "supportsHttpsTrafficOnly": true, 32 "minimumTlsVersion": "TLS1_2", 33 "allowBlobPublicAccess": false 34 } 35 }, 36 { 37 "type": "Microsoft.Storage/storageAccounts/blobServices/containers", 38 "apiVersion": "2025-06-01", 39 "name": "[concat(variables('storageAccountName'), '/default/', parameters('containerName'))]", 40 "dependsOn": [41 "[resourceId('Microsoft.Storage/storageAccounts', variables('storageAccountName'))]" 42], 43 "properties": { "publicAccess": "None" } 44 } 45], 46 "outputs": { 47 "storageAccountName": { 48 "type": "string", 49 "value": "[variables('storageAccountName')]" 50 } 51 } 52}

```
{ "$schema": "",
"contentVersion":
```

This template takes three input parameters — **location**, **storageAccountType** and **containerName**. The customer supplies them through the Azure portal UI, which is defined in **createUiDefinition.json**. Note that the **outputs** of the UI definition below match the **parameters** of the template above.

🟣 createUiDefinition.json

1{ 2 "$schema": "https://schema.management.azure.com/schemas/0.1.2-preview/CreateUIDefinition.MultiVm.json#", 3 "handler": "Microsoft.Azure.CreateUIDef", 4 "version": "0.1.2-preview", 5 "parameters": { 6 "basics": [], 7 "steps": [8 { 9 "name": "storageConfig", 10 "label": "Storage", 11 "elements": [12 { 13 "name": "storageAccountType", 14 "type": "Microsoft.Common.DropDown", 15 "label": "Storage account type", 16 "defaultValue": "Standard_LRS", 17 "toolTip": "Redundancy/performance tier for the demo storage account.", 18 "constraints": { 19 "allowedValues": [20 { "label": "Standard_LRS", "value": "Standard_LRS" }, 21 { "label": "Standard_GRS", "value": "Standard_GRS" }, 22 { "label": "Premium_LRS", "value": "Premium_LRS" } 23], 24 "required": true 25 } 26 }, 27 { 28 "name": "containerName", 29 "type": "Microsoft.Common.TextBox", 30 "label": "Blob container name", 31 "defaultValue": "demo", 32 "toolTip": "Container created inside the storage account.", 33 "constraints": { 34 "required": true, 35 "regex": "^[a-z0-9]([a-z0-9-]{1,61}[a-z0-9])$", 36 "validationMessage": "3-63 chars, lowercase letters, numbers and hyphens; must start and end with a letter or number." 37 } 38 } 39] 40 } 41], 42 "outputs": { 43 "storageAccountType": "[steps('storageConfig').storageAccountType]", 44 "containerName": "[steps('storageConfig').containerName]", 45 "location": "[location()]" 46 } 47

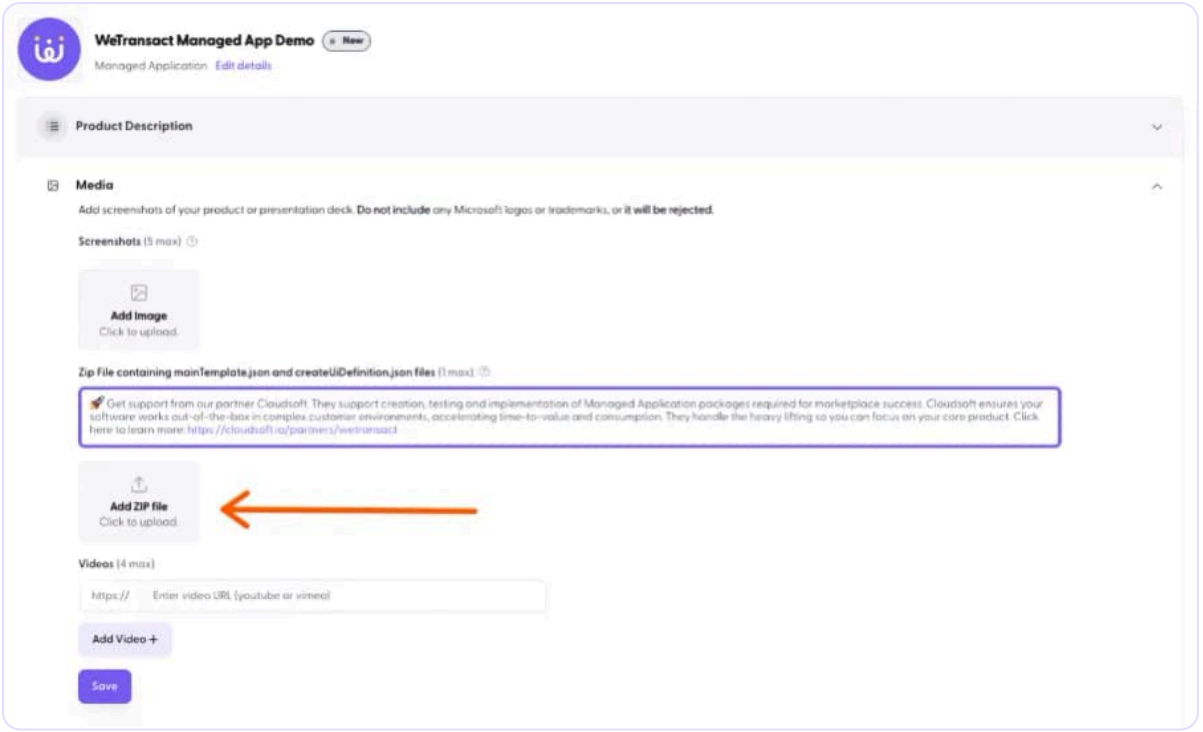
```
{ "$schema": "",
"handler":
```

Tip: If you do not have the in-house engineering capacity to build the package, WeTransact's partner Cloudsoft creates, tests and implements Managed Application packages. See cloudsoft.io/partners/wetransact.

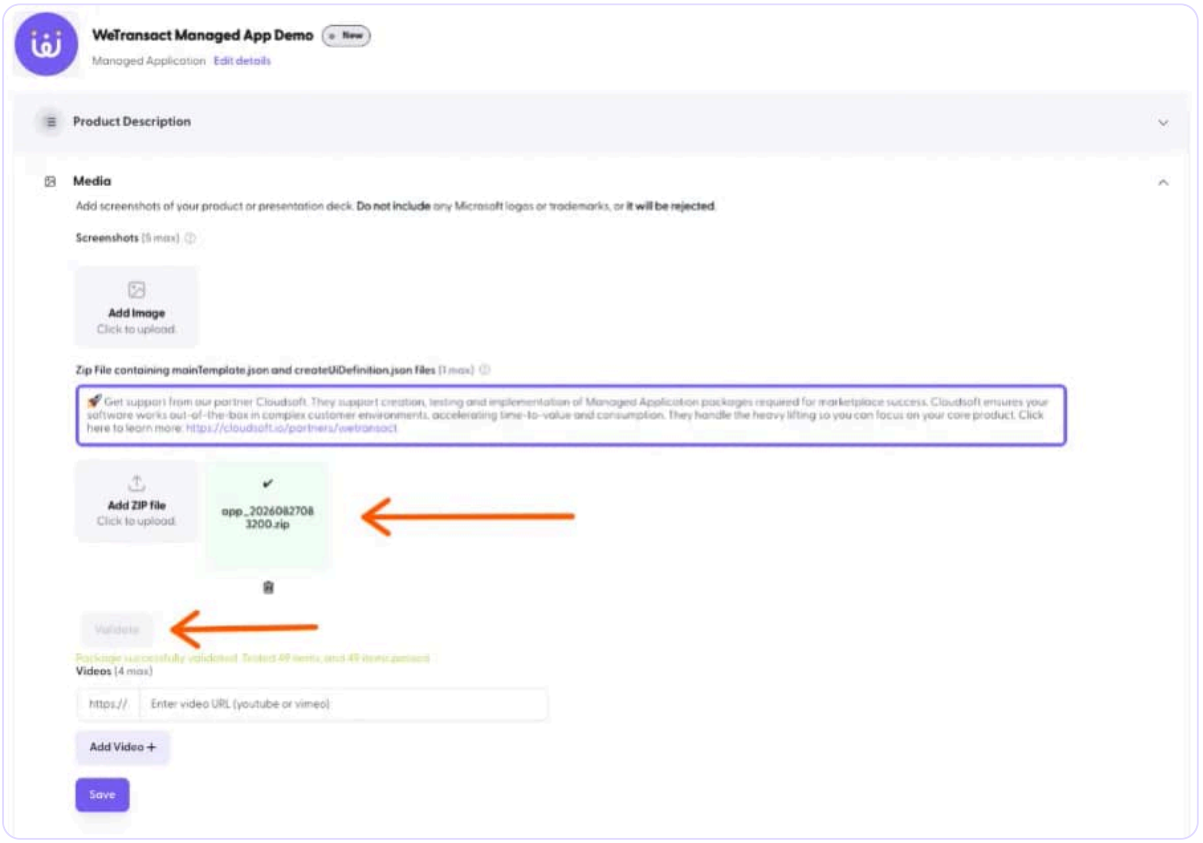
🟣 Uploading and publishing with WeTransact

1. In the WeTransact portal, create the Managed Application listing and complete the product description, media and plans — the same flow as any other listing.

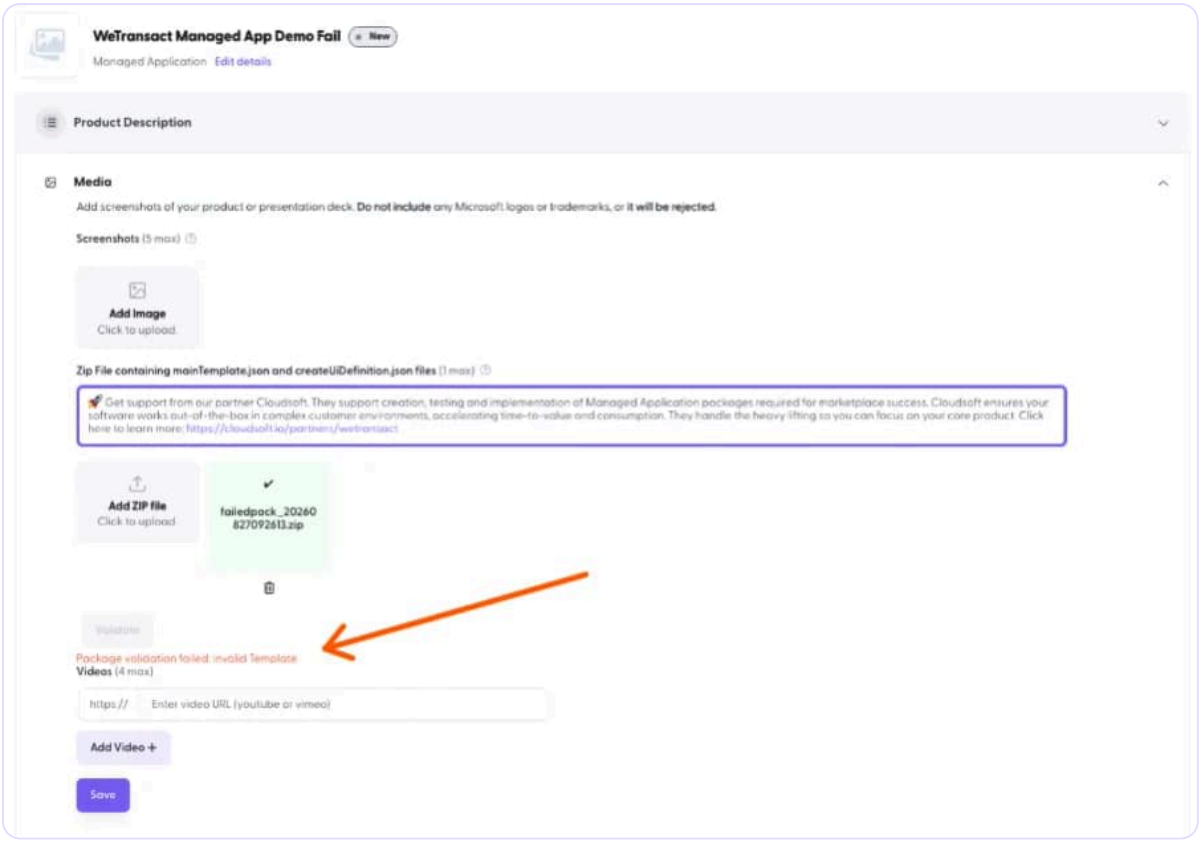
2. Upload the **.zip** package in the plan, in the field "***Zip file containing mainTemplate.json and createUiDefinition.json files***".
   [Screenshot: 02-portal-add-zip-file.png — the Add ZIP file control in the Media section of the listing]



Then click Validate.



If everything is good, the validation will succeed. If not, you will see an error message:



It's recommended to run the validation yourself as well.

3. **Publish.** Initial publication of a Managed Application offer typically takes up to 24 hours to go live, and republishing an update takes about as long. This is slower than SaaS because Microsoft performs a manual certification pass on the template files.***[TO CONFIRM — the 24-hour figure is WeTransact's observed experience, not a documented Microsoft SLA.]***

💬 ***For more, contact us at : support@wetransact.io***

Always-current version of this guide: docs.wetransact.io/build-managed-application-offer · Questions? Reply to this email and your CSM will pick it up.